

Ce que vous avez accompli

Module 1 : Rédaction avancée d'instructions

- ✓ Vous avez appris cinq modèles d'instructions réutilisables (identité, mission, méthodologie, limites, exemples few-shot)
- ✓ Vous avez créé des schémas indépendants du domaine avec des modèles et des [espaces réservés]
- ✓ Vous avez implémenté des exemples multidomains dans les secteurs de l'e-commerce, de l'éducation, de la finance et de la santé
- ✓ Vous avez structuré les instructions avec une organisation Markdown pour plus de clarté

Information clé : les instructions sont des spécifications complètes du comportement. Utilisez des schémas réutilisables pour créer des agents professionnels dans n'importe quel domaine.

Module 2 : Sortie structurée

- ✓ Vous avez défini des schémas Pydantic BaseModel pour obtenir une sortie JSON prévisible
- ✓ Vous avez utilisé output_key pour transmettre des données dans les workflows
- ✓ Vous avez compris l'exhaustivité du schéma : seuls les champs définis apparaissent dans la sortie
- ✓ Vous avez appris que la sortie structurée fonctionne à la fois pour les agents racine et les sous-agents intermédiaires

Information clé : la sortie structurée comble le fossé entre l'IA en langage naturel et l'intégration système. Le schéma est un contrat : définissez exactement ce dont vous avez besoin.

Module 3 : Choisir et configurer des modèles

- ✓ Vous avez choisi les modèles de manière stratégique : commencez par Pro pour le prototypage, puis optimisez avec Flash
- ✓ Vous avez configuré les paramètres de sécurité à l'aide de GenerateContentConfig et SafetySetting
- ✓ Vous avez utilisé des plages de température pour différentes tâches (0,0-0,3 pour les tâches factuelles, 0,8-1,0 pour les tâches créatives)
- ✓ Vous avez découvert les tarifs de Vertex AI et les schémas de sélection de modèles

Information clé : Commencez avec Gemini Pro pour le prototypage et pour établir des bases de qualité. Utilisez Flash pour optimiser les coûts et la rapidité après avoir effectué une analyse des déficits.

Module 4 : Planification de tâches complexes

- ✓ Vous avez activé le raisonnement en plusieurs étapes avec BuiltInPlanner et ThinkingConfig
- ✓ Vous avez compris les approches de planification par rapport aux approches multi-agents
- ✓ Vous avez comparé les résultats de la planification avec des exemples concrets (achat ou location d'une voiture)
- ✓ Vous avez appris quand utiliser BuiltInPlanner (Gemini) ou PlanReActPlanner (modèles

autres que Gemini)

Information clé : la planification transforme les agents réactifs en penseurs stratégiques. Utilisez BuiltInPlanner pour les modèles Gemini dotés de capacités de raisonnement. Des instructions simples suffisent, car la planification ajoute automatiquement le raisonnement structuré.

Applications concrètes

Grâce à ces compétences, vous pouvez désormais créer les éléments suivants :

Systèmes de service client

```
from pydantic import BaseModel, Field

class TicketOutput(BaseModel):
    ticket: dict = Field(description="Détails du ticket")
    priority: str = Field(description="Niveau de priorité")

LlmAgent(
    model="gemini-2.5-flash",
    instruction="[Persona détaillé avec limites]",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=True,
        thinking_budget=1024)
    ),
    output_schema=TicketOutput
)
```

Pipelines de traitement des données

```
from pydantic import BaseModel, Field
from typing import List

class AnalysisOutput(BaseModel):
    insights: List[str] = Field(description="Informations clés")
    metrics: dict = Field(description="métriques de performance")

LlmAgent(
    model="gemini-2.5-pro",
    instruction="[Framework d'analyse]",
    output_schema=AnalysisOutput
)
```

Automatisation du workflow

```
LlmAgent(
    model="gemini-2.5-flash",
    instruction="[Règles de processus]",
    output_key="approved",
```

```

generate_content_config=types.GenerateContentConfig(
    temperature=0.2,
    safety_settings=[...]
)
)

```

Assistants pédagogiques

```

LlmAgent(
    model="gemini-2.5-pro",
    instruction="[Méthodologie d'apprentissage]",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=False,
thinking_budget=2048)
    ),
    generate_content_config=types.GenerateContentConfig(temperature=0.5)
)

```

Checklist des bonnes pratiques

✓ Rédaction d'instructions

- Utiliser cinq modèles réutilisables : identité, mission, méthodologie, limites, exemples few-shot
- Créer des modèles génériques avec des [espaces réservés] pour plus de flexibilité au niveau du domaine
- Définir un persona clair avec une expertise spécifique
- Fixer des limites explicites (listes "Jamais"/"Toujours")
- Inclure deux ou trois exemples few-shot au format entrée → sortie
- Utiliser la structure Markdown pour améliorer la lisibilité

✓ Choix du modèle

- Commencer avec Gemini 2.5 Pro pour le prototypage et la définition des critères de qualité de référence
- Optimiser avec Gemini 2.5 Flash après l'analyse des déficits pour la production
- Configurer des paramètres de sécurité adaptés à votre audience
- Définir la température en fonction de la tâche : 0,0-0,3 (factuel), 0,4-0,7 (équilibré), 0,8-1,0 (créatif)

✓ Planification

- Activer BuiltInPlanner pour les modèles Gemini avec des tâches en plusieurs étapes
- Utiliser PlanReActPlanner pour les modèles autres que Gemini
- Configurer ThinkingConfig avec include_thoughts=True pour le débogage
- Définir thinking_budget de manière adaptée (entre 512 et 2 048 jetons en général)
- Appliquer des contraintes réalistes dans les instructions
- Utiliser une température plus basse (0,2-0,3) pour une planification systématique

✓ Sortie structurée

- Définir des schémas Pydantic BaseModel (pas des dictionnaires)
- Inclure TOUS les champs nécessaires dans le schéma (seuls les champs définis apparaissent)
- Utiliser Field(description=...) pour guider le LLM
- Utiliser output_key pour transmettre des données entre les agents
- Gérer les champs facultatifs avec Optional[T] et des valeurs par défaut
- Valider les résultats avant de les utiliser en production

Référence des schémas de code

Schéma 1 : Agent de service professionnel

```
from google.adk.agents import LlmAgent, BuiltInPlanner
from google.genai import types
from pydantic import BaseModel, Field

class ServiceResponse(BaseModel):
    response: str = Field(description="La réponse à l'utilisateur")
    action_required: bool = Field(description="Si une action est nécessaire")
    metadata: dict = Field(default={}, description="Métadonnées supplémentaires")

professional_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="service_pro",
    instruction="""\
PERSONA
Tu es [Nom], [Rôle] avec [Expérience].

# LIMITES
Jamais : [Liste des restrictions]
Toujours : [Liste des exigences]

# EXEMPLES
[Exemples few-shot]
\
\
\
\
""",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=True,
        thinking_budget=1024)
    ),
    output_schema=ServiceResponse,
    generate_content_config=types.GenerateContentConfig(
        temperature=0.5,
        safety_settings=[
            types.SafetySetting(
                category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
                threshold=types.HarmBlockThreshold.BLOCK_MEDIUM_AND_ABOVE
            )
        ]
    )
)
```

```
)  
)
```

Schéma 2 : Pipeline d'extraction de données

```
from pydantic import BaseModel, Field  
from typing import List  
from google.genai import types  
  
class Entity(BaseModel):  
    type: str = Field(description="Type d'entité")  
    value: str = Field(description="Valeur d'entité")  
    confidence: float = Field(description="Score de confiance 0-1")  
  
class ExtractionOutput(BaseModel):  
    entities: List[Entity] = Field(description="Entités extraites")  
    relationships: List[str] = Field(description="Relations identifiées")  
    metadata: dict = Field(default={}, description="Métadonnées d'extraction")  
  
extractor = LlmAgent(  
    model="gemini-2.5-flash",  
    name="data_extractor",  
    instruction="Extrais des données structurées à partir du texte",  
    output_schema=ExtractionOutput,  
    generate_content_config=types.GenerateContentConfig(  
        temperature=0.2 # Basse pour plus de cohérence  
    )  
)
```

Schéma 3 : Système de prise de décision

```
from google.genai import types  
  
decision_agent = LlmAgent(  
    model="gemini-2.5-pro",  
    name="decision_maker",  
    instruction="""  
Évalue les demandes conformément aux règles  
Prends en compte tous les facteurs et les dépendances.  
""",  
    planner=BuiltInPlanner(  
        thinking_config=types.ThinkingConfig(include_thoughts=False,  
        thinking_budget=2048)  
    ),  
    output_key="approved", # Sortie booléenne simple  
    generate_content_config=types.GenerateContentConfig(  
        temperature=0.1 # Très basse pour plus de cohérence  
    )  
)
```

Erreurs courantes à éviter

✗ Erreurs dans les instructions

- Personas vagues sans expertise spécifique ni connaissance du domaine
- Absence de limites entraînant des réponses inappropriées ou hors sujet
- Absence d'exemples few-shot, entraînant des schémas de comportement incohérents
- Instructions trop complexes qui déstabilisent le modèle
- Schémas spécifiques à un domaine plutôt que des modèles réutilisables

✗ Erreurs de configuration

- Démarrage avec Flash au lieu de Pro pour le prototypage (mauvaises bases)
- Utilisation de Pro pour des tâches simples en grande quantité après optimisation (surcoût injustifié)
- Paramètres de sécurité inadaptés à la cible (trop permissifs ou trop stricts)
- Température trop élevée pour des tâches factuelles (provoque des hallucinations)
- Absence d'analyse des écarts lors du passage de Pro à Flash

✗ Problèmes de planification

- Non-recours au planificateur pour des tâches complexes en plusieurs étapes
- Utilisation du mauvais type de planificateur (BuiltInPlanner pour les modèles autres que Gemini)
- Absence de contraintes réalistes rendant les plans irréalisables
- Surplanification de tâches simples en une seule étape
- Mauvaise configuration du paramètre `thinking_budget`
- Température élevée en planification (limite la pensée systématique)

✗ Problèmes de sortie

- Utilisation de dictionnaires au lieu de Pydantic BaseModel pour les schémas
- Absence de tous les champs nécessaires (seuls les champs définis apparaissent dans le résultat)
- Absence de `Field(description=...)` pour guider le LLM
- Aucune gestion des données facultatives/manquantes
- Incohérence dans les conventions d'attribution de noms
- Absence de vérification des résultats avant la mise en production

Communauté et assistance

Principales références

-  [Documentation sur les agents LLM d'ADK](#) : configuration et paramètres avancés
-  [Documentation sur le modèle Gemini](#) : capacités et fonctionnalités du modèle
-  [Tarification de Vertex AI](#) : optimisation des coûts et sélection de modèles
-  [Paramètres de sécurité de Gemini](#) : guide de configuration de la sécurité
-  [Raisonnement Gemini](#) : capacités de planification

Ressources

De la communauté

- [Implémenter des modèles de conception d'agents d'Anthropic avec Google ADK](#) : apprenez à appliquer des modèles de conception d'agents et des techniques d'instruction professionnels à l'aide d'ADK, y compris la création de personas et la conception de comportements.
- [Créer des applications agentiques avec Google ADK](#) : guide pratique pour la création d'agents prêts pour la production avec des sorties structurées et une intégration en environnement réel.
- [S'amuser avec les workflows agentiques dans ADK](#) : explorer les capacités de planification et le raisonnement en plusieurs étapes avec des modèles d'orchestration de workflows.
- [Développement pratique d'agents IA avec ADK](#) : bonnes pratiques pour le développement d'agents, y compris la configuration, les instructions et les stratégies de déploiement.
- [Accélérez votre parcours avec l'IA agentique grâce à l'ADK de Google – du code au cloud](#) : un guide complet et pratique qui vous accompagne tout au long de la création d'agents de base, de la configuration de Gemini et du déploiement dans le cloud.

Des questions ? Posez-les sur le [forum de la communauté](#).

Mémento

Stratégie de sélection de modèle

1. Commencez avec Gemini 2.5 Pro : prototypage, bases de qualité, raisonnement complexe
2. Optimisez avec Gemini 2.5 Flash : production, volume élevé, optimisation des coûts
3. Effectuez systématiquement l'analyse des déficits lors du passage Pro → Flash

Guide des températures

0,0 à 0,3 : faits, données extraction, analyse, cohérence
0,4 à 0,7 : aide générale et équilibrée, service client
0,8 à 1,0 : écriture créative, brainstorming, marketing

Seuils de sécurité

BLOCK_LOW_AND_ABOVE : niveau le plus strict - enfants, grand public
BLOCK_MEDIUM_AND_ABOVE : niveau standard - usage, professionnel et général
BLOCK_ONLY_HIGH : niveau souple - recherche, outils internes

Quel outil dois-je utiliser et quand ?

BuiltInPlanner : modèles Gemini, tâches en plusieurs étapes, dépendances
PlanReActPlanner : modèles autres que Gemini avec planification structurée
output_schema : Pydantic BaseModel pour des JSON structurées
output_key : transfert de données entre agents, extraction simple de valeurs

Schémas d'instructions

Identité : [Nom], [Rôle], [Expertise]
Mission : rôle et fonction de l'agent
Méthodologie : mode de fonctionnement de l'agent
Limites : listes Jamais/Toujours
Few-shot : exemples d'entrée/sortie

Conclusions

Vous êtes désormais capable de passer d'agents simples à des systèmes d'IA sophistiqués.
Vos agents peuvent désormais effectuer les actions suivantes :

- Réfléchir différemment et respecter les limites
- Planifier des solutions complexes en plusieurs étapes
- Produire des données structurées prêtes à l'usage
- Fonctionner de manière sûre et efficace

Les bases sont établies.